Memory Management in xv6

Implementation report: virtual memory, accounting, page tables, and demand paging

Scope

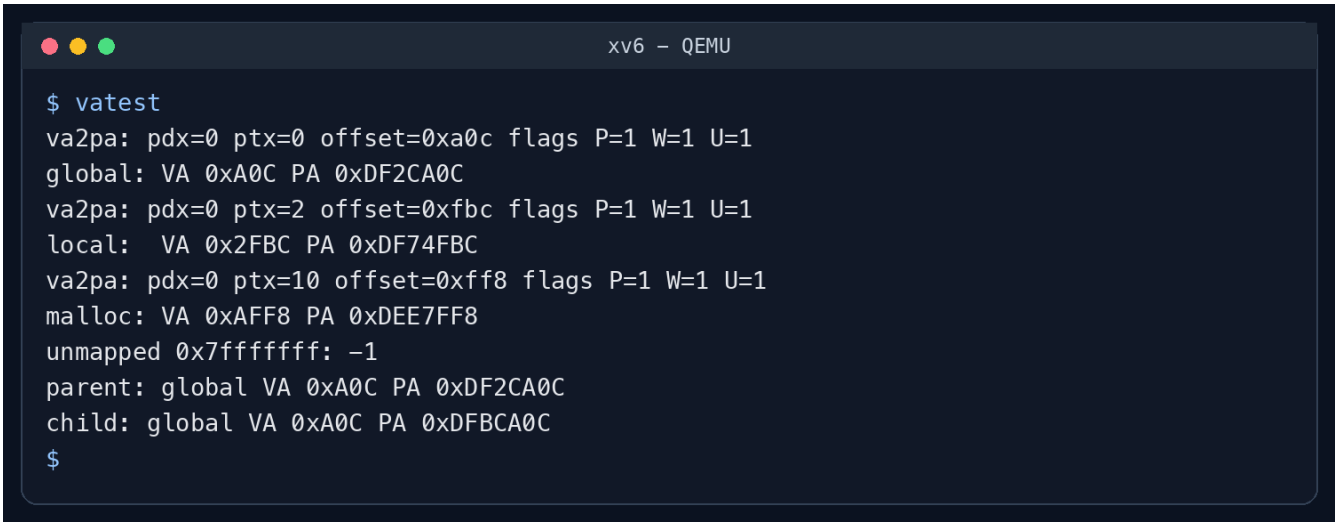
The implementation extends the x86 xv6 kernel with three observation system calls and lazy heap allocation. It preserves the original two-level paging model and keeps all policy decisions inside the kernel.

1. Virtual-to-physical address translation

The va2pa system call accepts a virtual address from the calling process. The kernel uses walkpgdir without allocating a page table, rejects addresses without both PTE_P and PTE_U, and combines PTE_ADDR with the 12-bit page offset. It also prints the directory index, table index, offset, and P/W/U flags.

- Global, stack-local, and malloc addresses translate successfully.
- 0x7fffffff returns -1 because no user-accessible mapping exists.
- fork copies user frames, so parent and child retain the same VA but receive different PAs.

Observed output



Physical memory and page-table visibility

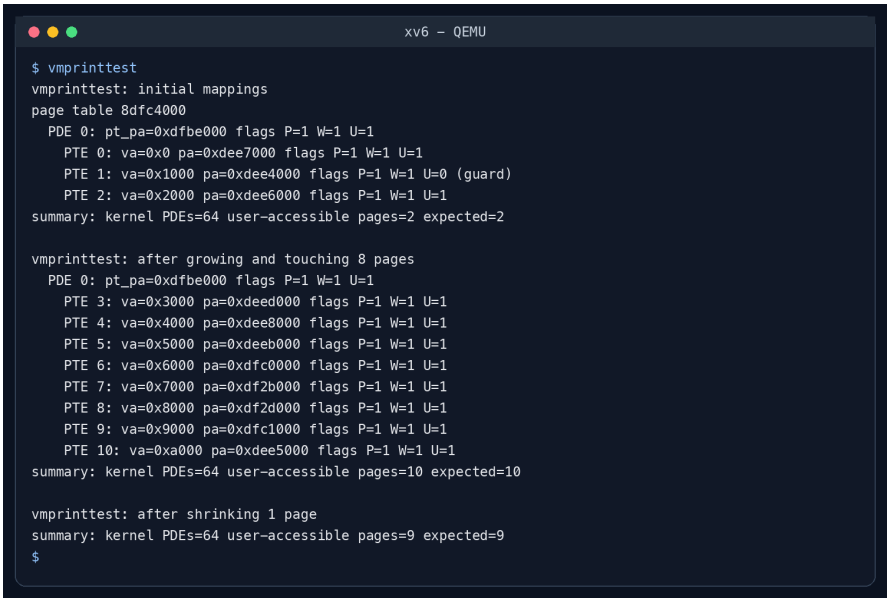
2. Physical memory statistics

kalloc.c maintains free_pages and total_pages. Updates to the free count occur while holding kmem.lock. getmeminfo returns allocator totals, sz / PGSIZE for the process address space, and the page-directory plus allocated page-table pages. The test confirms that frames consumed by fork are recovered after wait.



3. Hierarchical page-table dump

vmprint walks every present user-side PDE and then its present PTEs. Each line reports indexes, virtual and physical addresses, and P/W/U flags. Kernel PDEs are counted but not expanded. A mapped entry with U=0 is labelled as the exec-created stack guard page. The reported user-page count matches sz / PGSIZE minus that guard.



Lazy heap allocation

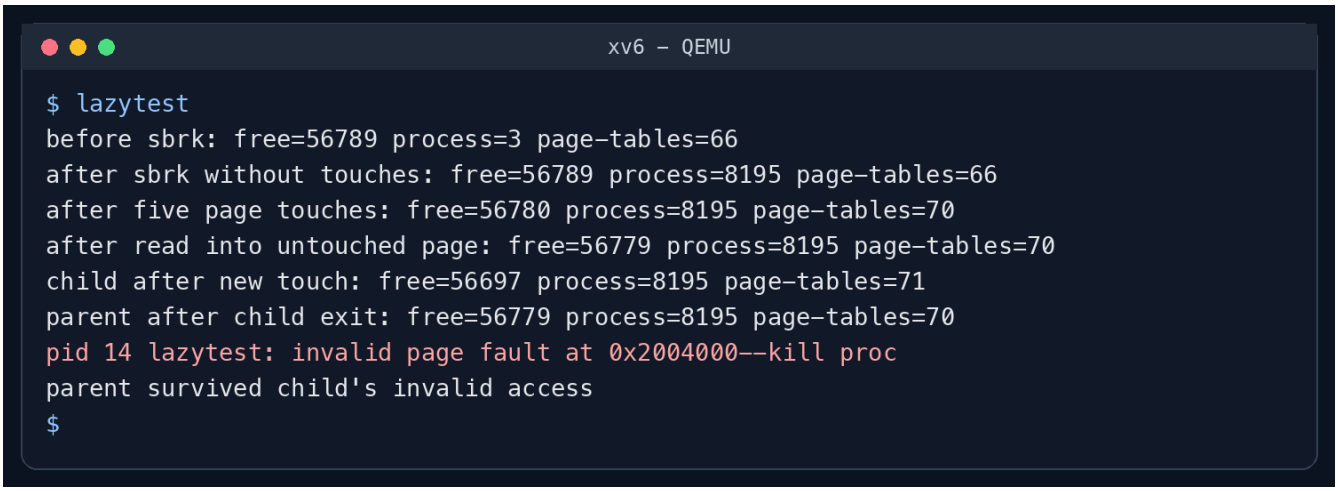
4. Demand paging design

Positive sbrk requests now reserve virtual address space by increasing proc.sz without allocating frames. The process records heap_start at exec time, preventing lazy allocation from treating code, the guard page, or the stack as heap. Negative sbrk requests still call deallocuvm, which safely skips absent lazy mappings.

- trap.c handles only not-present page faults in [heap_start, sz). A valid fault allocates and zeroes one frame, maps it with PTE_W | PTE_U, and resumes execution.
- copyuvm skips absent entries so fork preserves sparse heaps instead of panicking.
- Argument validation and copyout materialize valid lazy pages before the kernel accesses user buffers. This allows read and write on untouched heap pages without a kernel fault loop.
- Out-of-memory and out-of-range faults mark only the offending process as killed.

Key observation

Reserving 32 MB changes process_pages from 3 to 8195 while free_pages remains 56789. Physical memory is consumed only after individual pages are touched or used as a syscall buffer.



Verification and conclusions

5. Measured results

Scenario	Free	Process	Page tables
Q2 before allocation	56789	3	66
Q2 after four touches	56785	7	66
Q2 while child exists	56711	7	66
Q2 after child reaped	56785	7	66
Q4 after 32 MB sbrk	56789	8195	66
Q4 after five touches	56780	8195	70
Q4 after syscall buffer	56779	8195	70
Q4 after child exits	56779	8195	70

6. Problems and resolutions

Lazy allocation versus vmprint

The page-table test touches the eight reserved pages before its second dump, so the new mappings remain visible under demand paging.

Kernel access to lazy buffers

System-call pointer validation allocates valid missing heap pages before file and pipe code dereferences them.

Sparse fork

copyuvm treats missing PTEs as intentional holes and copies only present mappings.

Modern cross-toolchain

The Makefile detects i686-elf tools and suppresses baseline xv6 warnings that newer GCC versions promote to errors.

7. Validation

The kernel, filesystem image, and all four assignment programs compile successfully. Each assignment test was executed under QEMU. The standard xv6 usertests suite also completed with ALL TESTS PASSED. The final source package contains only modified kernel interfaces, user interfaces, the Makefile, and the four tests.